RAG Agent Level 4 — Portfolio Upgrade

Production-oriented RAG + Agent platform with evaluation, observability, cost controls, and CI quality gates

Based on the uploaded project specification. The original design already includes RAG retrieval, agent routing, tracing, a 50-question evaluation set, metrics, and cost reporting. This document upgrades that architecture into a stronger portfolio project without changing its core seams.

1. What the upgraded project should demonstrate

Capability	Upgrade	Portfolio signal
RAG	Hybrid retrieval: dense embeddings + BM25/keyword retrieval, followed by reranking	Shows modern retrieval engineering
Agent	Explicit tool-routing policy, structured tool schemas, retries and timeouts	Shows reliable agent design
Evaluation	Automated retrieval + answer quality gates with regression tracking	Shows measurable AI quality
Observability	Langfuse/LangSmith-compatible traces with latency, tokens, cost and errors	Shows production thinking
Cost	Per-query cost attribution, budgets and model-routing policy	Shows optimization
Deployment	Dockerized API + dashboard + health checks	Shows end-to-end engineering
CI/CD	Evaluation suite runs on every relevant pull request	Shows engineering maturity

2. Target architecture

```
User
  |
  v
FastAPI / API Gateway
  |
  +--> Request validation + auth/rate limit
  |
  v
Agent Orchestrator
  |--------------------|
  |                    |
  v                    v
RAG Pipeline         Tool Router
  |                    |
  +--> Query rewrite   +--> Calculator / domain tools
  +--> Hybrid retrieve +--> timeout / retry / validation
  +--> Reranker
  +--> Context builder
  |
  v
LLM / Response Generator
  |
  +--> citations + structured answer
  |
  v
Trace + Evaluation + Cost layer
  |
  +--> Langfuse/LangSmith
  +--> offline eval results
  +--> dashboard / alerts

Data:
documents -> chunking -> embeddings -> vector DB
                     -> BM25 index
```

3. Recommended repository structure

```
rag_agent_level4/
■■■ api/
■   ■■■ main.py
■   ■■■ schemas.py
■   ■■■ middleware.py
■■■ src/
■   ■■■ agent.py
■   ■■■ retriever.py
■   ■■■ reranker.py
■   ■■■ llm.py
■   ■■■ tools.py
■   ■■■ prompts.py
■   ■■■ tracer.py
■   ■■■ cost.py
■   ■■■ config.py
■■■ ingestion/
■   ■■■ loader.py
■   ■■■ chunker.py
■   ■■■ index.py
■■■ evals/
■   ■■■ eval_questions.json
■   ■■■ retrieval_metrics.py
■   ■■■ answer_metrics.py
■   ■■■ regression_gate.py
■   ■■■ run_eval.py
■■■ dashboard/
■   ■■■ app.py
■■■ tests/
■   ■■■ test_retrieval.py
■   ■■■ test_agent.py
■   ■■■ test_tools.py
■■■ docker/
■■■ .github/workflows/eval.yml
■■■ docker-compose.yml
■■■ README.md
```

4. Evaluation upgrade

The original project already evaluates 50 questions and measures Precision@k, Faithfulness, Answer Relevancy and Keyword Recall. The upgraded version should preserve those metrics and add retrieval and regression controls.

Layer	Metric / test	Gate example
Retrieval	Recall@k	≥ 0.90
Retrieval	MRR / nDCG	Track baseline + regression threshold
Answer	Faithfulness	≥ 0.90
Answer	Answer Relevancy	≥ 0.85
Answer	Citation correctness	≥ 0.95
Agent	Tool-selection accuracy	≥ 0.95
Reliability	Malformed-tool-call rate	≤ 1%
Cost	Average $ / query	Budget threshold
Latency	p95 latency	SLO threshold

Important: the thresholds above are proposed portfolio-quality gates, not results from the uploaded project. They should be tuned after establishing a real baseline.

5. Observability design

Keep the original Trace/Span model, but make it compatible with a hosted tracing backend. Each request should expose a single trace containing retrieval, reranking, tool calls, LLM generation, latency, token usage, cost and errors.

Span	Capture
retrieval	query, top-k docs, scores, retrieval latency
rerank	candidate count, reranked docs, latency
tool_call	tool name, validated args, result, retries, latency
llm_generation	model, prompt/completion tokens, latency, cost
response	answer, citations, quality metadata

6. Agent reliability upgrades

• **Structured tool calls:** validate every tool argument against a schema before execution.

• **Timeouts:** every external tool call has a bounded execution time.

• **Retries:** retry only transient failures and cap the retry count.

• **Fallbacks:** if retrieval confidence is low, return a transparent uncertainty response instead of fabricating.

• **Prompt boundaries:** retrieved documents are data, not instructions; explicitly separate context from agent instructions.

• **Auditability:** every tool decision and retrieval result is present in the trace.

7. Cost and performance dashboard

Extend the existing cost dashboard from total/average tokens and projected cost per 1,000 queries into an operational dashboard.

Dashboard panel	Purpose
Cost / 1,000 queries	Compare model and retrieval configurations
p50 / p95 latency	Identify slow retrieval, tools or generation
Token distribution	Detect unexpectedly long prompts/answers
Retrieval hit rate	Monitor retrieval quality
Faithfulness trend	Catch answer-quality regressions
Tool error rate	Monitor agent reliability
Worst queries	Quickly identify cases for debugging

8. CI/CD quality gate

```
Pull Request
   |
   +--> unit tests
   +--> lint / type checks
   +--> build Docker image
   +--> run evaluation suite
   +--> compare against baseline
   |
   +--> PASS: merge
   |
   +--> FAIL: publish metric regression report
```

A strong portfolio implementation should show that a prompt, retriever, model, or agent change cannot silently degrade the system. The CI job should publish the evaluation summary and highlight regressions.

9. Production-readiness checklist

■ Environment-based configuration and secrets management

■ Health/readiness endpoints

■ Structured JSON logging

■ Request IDs and trace IDs

■ Rate limiting

■ Input/output validation

■ Document ingestion versioning

■ Evaluation dataset versioning

■ Automated regression gates

■ Docker Compose local deployment

■ API documentation

■ Failure and fallback behavior documented

10. Portfolio README positioning

Recommended project title: Production-Oriented RAG Agent with Automated Evaluation & Observability

One-line pitch: Built an end-to-end RAG agent platform that combines hybrid retrieval, tool routing, automated quality evaluation, distributed-style tracing, per-query cost attribution, and CI regression gates.

Best demo flow: Ask a knowledge-base question → show retrieved sources → show agent/tool decision → show final cited answer → open trace → show latency/tokens/cost → run evaluation dashboard → demonstrate a failing quality gate.

11. Upgrade priority

Priority	Build first	Why
P0	Real LLM + robust retrieval + citations	Makes the demo genuinely useful
P0	Automated evaluation + CI regression gate	Strongest differentiator
P0	Tracing with latency/token/cost attribution	Makes behavior explainable
P1	Hybrid retrieval + reranking	Modernizes retrieval
P1	Dashboard	Makes results immediately visible
P1	Docker/API deployment	Demonstrates full-stack engineering
P2	Authentication/rate limiting	Adds production hardening

Source basis: uploaded RAG + Agent Eval Demo specification. The original project specifies a 10-document corpus, TF-IDF retrieval, mock LLM, calculator tool, tracing, 50-question evaluation, quality metrics, and cost dashboard; this document proposes upgrades around those existing components. ■filecite■turn0file0■L10-L34■